CÓDIGOCLARO · MATERIAL DE ESTUDIO

XML, XPath, JSON y MongoDB

Del documento estructurado al intercambio HTTP y las consultas de documentos

1. XML bien formado y estructura

Objetivo: Reconocer la sintaxis que hace que un documento XML pueda analizarse antes de validar su significado.

XML organiza información en elementos anidados con etiquetas de inicio y fin, atributos entre comillas y un único elemento raíz. Los nombres distinguen mayúsculas de minúsculas. Un elemento vacío puede escribirse como etiqueta autocerrada. Los caracteres reservados deben escaparse en texto y atributos: por ejemplo, el ampersand se escribe como entidad. El prólogo puede declarar la versión y codificación, pero no reemplaza la estructura. Bien formado significa que se cumple la sintaxis; no demuestra que el documento responda al modelo de negocio. Para intercambiar XML, acuerda además los nombres, unidades, cardinalidades y reglas con quienes producen y consumen el documento.

Código · ejemplo

```
[XML]
<?xml version="1.0" encoding="UTF-8"?>
<curso>
  <alumno id="a17">
    <nombre>Ana &amp; Luis</nombre>
    <nota>8.5</nota>
  </alumno>
</curso>
```

Práctica: Indica qué error sintáctico aparece si se omite `</alumno>` y diferencia bien formado de válido.

Respuesta razonada: El elemento alumno queda sin cerrar y el analizador debe rechazar el documento. Una estructura bien formada puede no ser válida frente a un esquema, por ejemplo si falta un campo obligatorio. Primero se comprueba sintaxis; después, si el contrato lo exige, se valida contra XSD u otra gramática.

2. Elementos, atributos y texto

Objetivo: Elegir una representación XML clara y estable para datos simples y relaciones entre entidades.

Los elementos describen estructura y pueden contener otros elementos o texto; los atributos suelen expresar identificadores o propiedades breves de un elemento. XML no obliga a una única decisión, así que el contrato debe fijarla. Mezclar texto significativo con hijos puede dificultar transformaciones, y los nombres no deben codificar datos variables. Conserva la distinción entre contenido y metadatos, evita depender del orden de atributos y define cómo representar valores ausentes, vacíos y desconocidos. Una colección puede repetirse mediante elementos hermanos. Tipos como fecha y decimal requieren reglas externas o un esquema; el texto `8.50` por sí solo no impone una escala numérica.

Código · ejemplo

```
[XML]
<pedido id="p42">
  <cliente>c9</cliente>
  <lineas>
    <linea sku="libro-3" cantidad="2"/>
  </lineas>
</pedido>
```

Práctica: Diseña una regla para representar varios artículos y distingue colección vacía de dato omitido.

Respuesta razonada: Repite `linea` dentro de `lineas` y documenta si un contenedor vacío significa que no hay artículos. Si falta `lineas`, el emisor quizá desconoce el dato o no lo envió; esa diferencia solo tiene significado si el contrato la define. Identificadores estables van bien como atributos, pero no son automáticamente claves de base de datos.

3. Espacios de nombres

Objetivo: Evitar colisiones entre vocabularios XML mediante URI de espacio de nombres y prefijos de consulta.

Un espacio de nombres asocia nombres expandidos con una URI; el prefijo es solo un alias local, no una etiqueta global fija. Dos documentos pueden usar prefijos distintos y representar el mismo nombre expandido si la URI coincide. Los atributos sin prefijo no heredan el espacio de nombres predeterminado de los elementos. Al consultar XML, el programa suele enlazar un prefijo propio a la URI correcta; no debe asumir que el documento usa el mismo prefijo elegido por quien consulta. Una URI de namespace identifica un vocabulario, pero no necesariamente es una página que se deba descargar. Esta regla es una fuente frecuente de selecciones XPath vacías.

Código · ejemplo

```
[XML]
<c:curso xmlns:c="urn:escuela:curso">
  <c:alumno codigo="a17">Ana</c:alumno>
</c:curso>
```

Práctica: Explica por qué una consulta `//alumno` sin namespace puede devolver cero resultados.

Respuesta razonada: El nombre expandido del elemento es `{urn:escuela:curso}alumno`, no el nombre sin namespace. Registra, por ejemplo, el prefijo `e` con esa URI en la API de consulta y busca `//e:alumno`; el alias puede diferir del prefijo `c` del archivo. El atributo `codigo` está sin namespace en este ejemplo.

4. XPath: rutas y selección

Objetivo: Construir rutas XPath para seleccionar nodos concretos y distinguir ruta absoluta de relativa.

XPath navega un árbol XML mediante pasos, ejes y pruebas de nodo. La ruta desde `/` empieza en la raíz del documento; `//` permite buscar descendientes en cualquier profundidad y puede ser menos precisa. Una expresión relativa depende del nodo de contexto. Los nombres con prefijo deben resolverse con el namespace del vocabulario. Seleccionar nodos produce una secuencia o conjunto según la versión y la biblioteca; convertir ese resultado a texto es una decisión aparte. Especifica qué hacer con cero, uno o varios resultados. Una consulta legible reduce errores cuando cambia un documento y facilita probar casos de ausencia, repetición y anidamiento.

Código · ejemplo

```
[XPath 3.1]
/c:curso/c:alumno[@codigo = 'a17']/text()
```

Práctica: Propón una consulta para seleccionar los elementos `linea` de todos los pedidos y considera el namespace.

Respuesta razonada: Si los pedidos son la raíz de consulta, usa una ruta relativa como `//p:pedido/p:lineas/p:linea` con `p` enlazado a la URI declarada. La consulta devuelve nodos, no necesariamente una sola cadena. Si el documento cambia de vocabulario o un nombre aparece en otro namespace, ajusta el vínculo y los pasos en vez de quitar prefijos a ciegas.

5. Predicados y posición

Objetivo: Usar predicados XPath para filtrar elementos por atributo, texto o posición con el contexto correcto.

Un predicado entre corchetes filtra los nodos seleccionados. Dentro del predicado, `@id` accede a atributos y `text()` a nodos de texto. El significado de una posición depende del conjunto de nodos sobre el que opera cada paso y, en ejes inversos, del orden del eje; anidar predicados puede cambiar qué se cuenta. Comparar cadenas y números requiere saber cómo la versión XPath convierte valores. No uses posiciones como identificadores permanentes si el documento puede reordenarse. Prueba con cero coincidencias, una coincidencia, varios resultados y atributos ausentes; revisa también espacios en blanco cuando el contrato no los normalice.

Código · ejemplo

```
[XPath 3.1]
/c:curso/c:alumno[@activo = 'true'][1]
```

Práctica: Compara la intención de seleccionar el primer alumno activo con el primer alumno si está activo.

Respuesta razonada: La expresión filtra alumnos activos y después toma el primero de esos resultados. Si se aplicara `[1]` antes de la condición, podría examinar solo el primer alumno y no hallar ninguno aunque exista otro activo. Escribe la intención en palabras y valida el orden de pasos con un XML de prueba donde el primer elemento no cumpla la condición.

6. JSON y su modelo de datos

Objetivo: Leer y producir JSON conforme al formato estándar sin confundir texto JSON con objetos del lenguaje.

RFC 8259 define valores JSON como objeto, array, cadena, número, booleano o null. Los objetos contienen pares nombre-valor y los arrays conservan orden. JSON no tiene comentarios, fechas, NaN ni referencias compartidas; fecha y dinero necesitan convenciones acordadas. Un nombre duplicado dentro del mismo objeto produce resultados impredecibles entre analizadores, así que evítalo. La sintaxis usa comillas dobles y escapes; una coma final no es válida. Al analizar JSON, valida estructura y límites de tamaño, y al serializar respeta la codificación UTF-8. El analizador puede construir objetos del lenguaje, pero no conserva necesariamente el formato original ni el orden textual de todos los detalles.

Código · ejemplo

```
[JSON]
{"id":"p42","items":[{"sku":"libro-3","quantity":2}],"paid":false}
```

Práctica: Identifica cómo representar una fecha y un campo ausente sin inventar tipos JSON.

Respuesta razonada: Una convención común intercambia fechas como cadenas en formato documentado, por ejemplo ISO 8601, y se valida al recibirlas. Un campo ausente no es lo mismo que `null`: el contrato debe decir si se omite, se desconoce o se declara explícitamente vacío. Evita duplicar claves y verifica que `quantity` sea un entero positivo donde el dominio lo exija.

7. JSON en una respuesta HTTP

Objetivo: Relacionar JSON con HTTP, estado de respuesta, tipo de contenido y validación del lado cliente.

JSON es una representación; HTTP define cómo se solicita y responde a un recurso. El servidor debe enviar un estado adecuado y un `Content-Type` como `application/json`; los clientes no deben inferir el éxito solo porque el cuerpo se pudo analizar. `fetch` puede resolver la promesa incluso ante respuestas 4xx o 5xx, de modo que se comprueba `response.ok` y después se trata la lectura del cuerpo, que también puede fallar. La validación de datos se hace tanto en servidor como en cliente: lo enviado por el navegador puede manipularse. No incluyas secretos ni datos que el usuario no esté autorizado a ver en una respuesta.

Código · ejemplo

```
[JavaScript]
const response = await fetch('/api/pedidos/p42');
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const pedido = await response.json();
```

Práctica: ¿Qué control falta si se asume que la respuesta analizada ya es un pedido correcto?

Respuesta razonada: Falta verificar que el contenido cumpla el esquema del pedido y que las operaciones puedan tolerar campos ausentes o datos incorrectos. El estado HTTP informa del resultado de la petición, no certifica el dominio del JSON. El servidor aplica autorización y valida entradas; el navegador puede validar adicionalmente para mostrar errores útiles.

8. MongoDB: documentos y colecciones

Objetivo: Comprender documentos BSON, colecciones e identificadores al modelar datos de aplicación.

MongoDB almacena documentos BSON agrupados en colecciones. Cada documento tiene un `_id` único en su colección, que puede generar el controlador si no se proporciona. Los documentos permiten subdocumentos y arrays, lo que facilita leer juntos datos que suelen consultarse juntos. La decisión de incrustar o referenciar depende de cardinalidad, tamaño, patrón de acceso y actualización: un array ilimitado puede crecer sin control. Diseña consultas antes que el esquema, pero conserva una sola fuente de verdad para datos que cambian independientemente. Los tipos BSON incluyen valores que JSON textual no representa de la misma manera; al cruzar una API, define una representación explícita para identificadores y fechas.

Código · ejemplo

```
[MongoDB]
{ _id: "p42", customerId: "c9", lines: [ { sku: "libro-3", quantity: 2 } ] }
```

Práctica: ¿Cuándo conviene incrustar `lines` y qué límite revisarías?

Respuesta razonada: Incrústalas si pertenecen al pedido, se leen juntas y su crecimiento está acotado. Si la colección de líneas puede crecer indefinidamente o se actualiza y consulta de forma independiente, evalúa documentos separados y referencias. Revisa tamaño del documento, duplicación, consistencia de escrituras y necesidades de lectura; no elijas solo por la forma visual del JSON.

9. CRUD, filtros e índices

Objetivo: Leer y modificar documentos mediante filtros precisos y apoyar las consultas frecuentes con índices.

Las operaciones CRUD crean, leen, actualizan y eliminan documentos. Un filtro vacío puede afectar a todos los documentos: antes de modificar o borrar, confirma el predicado y limita la operación a la identidad esperada. La proyección reduce los campos devueltos; evita exponer información sensible. Un índice puede acelerar filtros y ordenaciones, con coste adicional en almacenamiento y en cada escritura. El orden de campos en índices compuestos importa según el patrón de consulta. Prueba consultas con datos representativos y revisa el plan de ejecución; evita crear índices a ciegas. En una aplicación, deja que el controlador parametrice valores en vez de construir cadenas de consulta con entrada sin validar.

Código · ejemplo

```
[MongoDB Query API]
db.orders.find({ customerId: "c9" }, { _id: 1, status: 1 })
```

Práctica: Diseña un índice para filtrar por cliente y ordenar por fecha descendente.

Respuesta razonada: Un índice candidato es `{ customerId: 1, createdAt: -1 }`, siempre que la consulta real filtre por cliente y ordene por fecha. Confirma con el plan de ejecución y datos de volumen similar; el índice consume espacio y ralentiza inserciones y actualizaciones. Evita un filtro accidental `{}` y limita la proyección a campos necesarios.

10. Agregación y validación del esquema

Objetivo: Componer transformaciones de documentos y proteger la colección con reglas de validación.

Una canalización de agregación procesa documentos por etapas: por ejemplo, filtrar con `$match`, descomponer arrays con `$unwind`, agrupar con `$group` y ordenar con `$sort`. Colocar un filtro temprano puede reducir trabajo, aunque debe preservar la semántica. Los valores nulos, campos ausentes y tipos heterogéneos pueden cambiar los cálculos. La validación de esquema puede rechazar o advertir ante documentos que incumplen reglas, pero no reemplaza la validación de negocio ni la autorización. Ajusta validadores gradualmente al migrar datos heredados y observa errores de escritura. Documenta la versión del servidor y controlador, porque algunas opciones disponibles dependen de esas versiones.

Código · ejemplo

```
[MongoDB aggregation]
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $unwind: "$lines" },
  { $group: { _id: "$lines.sku", units: { $sum: "$lines.quantity" } } }
])
```

Práctica: Predice qué ocurre con pedidos no pagados y con pedidos pagados sin líneas.

Respuesta razonada: `$match` elimina los pedidos no pagados antes de expandir líneas. `$unwind` normalmente descarta arrays vacíos o ausentes si no se configura para preservarlos; por tanto, esos pedidos no aportan al resultado. Para incluirlos, revisa `preserveNullAndEmptyArrays` y define cómo debe contar una línea ausente. La respuesta final depende del contrato de negocio y se comprueba con ejemplos frontera.

Fuentes oficiales

W3C · XML 1.0 · W3C · Namespaces in XML · W3C · XPath 3.1 · IETF · RFC 8259 JSON · MongoDB · Manual · MongoDB · Aggregation pipeline · MongoDB · Schema validation